VibeGuard Compiler：コンパイラ最適化における セキュリティ性質消失の追跡と第一喪失点の特定

近藤 悠太*

概要：生成 AI によるコード生成の普及により，レビューと静的解析を通過したソースコードがそのまま出荷成果物になるとは限らない．ソース上の秘密消去や認可検査は，最適化水準やビルド構成によって成果物から失われうる．本稿では，コンパイラのソース改変と再ビルドを伴わず，配布されたコンパイラの公式拡張機構とコマンドラインフラグのみでビルド工程を観測し，性質ごとに宣言した観測述語が最初に偽になった工程（第一喪失点）を特定して，その根拠を検証可能な証拠束として成果物に付随させる手法を提案する．第一喪失点の Pass 帰属は，Pass 単位の IR ダンプ列の走査・実行 Pass 数の二分探索・out-of-tree な Pass プラグインという相補的な三観測チャネルの一致で判定し，観測が生成物を変えないこと（バイト単位一致）を測定ごとに証拠へ記録する．5 性質×2 コンパイラ×4 最適化水準×緩和フラグの 80 セル評価では，memset による秘密消去の消失が Dead Store Elimination に帰属される一方，assert による認可検査と条件付きコンパイルの防御は最適化の実行前に既に消えており，Pass 単位の帰属では説明できない．三法一致で Pass まで帰属が確定したのは clang-18 の 8 セルである．あわせて，シンボル名の検索が消失判定のオラクルとして無効であることを対照実験で示し，ゼロ化命令に基づく測定手順を与える．生成した証拠は，生成器の正準化器を用いない検証器で照合でき，成果物と証拠の不整合を検出できる．

キーワード：コンパイラ最適化，セキュリティ性質の消失，デッドストア除去，工程帰属，独立検証

VibeGuard Compiler: Tracking Where Security Properties Are Lost in Compilation and Locating First-Observed Loss Points

Yuta Kondo*

Abstract: With the spread of AI-assisted code generation, code that passes review and static analysis is not necessarily what ships: secret erasure and authorization checks can be lost from the artifact depending on optimization levels and build configuration. Without modifying or rebuilding the compiler, our method observes the build process of an off-the-shelf compiler through official extension mechanisms and command-line flags, identifies the stage at which a per-property observation predicate first becomes false, and seals the grounds into a verifiable evidence bundle attached to the artifact. Pass-level attribution requires agreement of three complementary observation channels and was reached for 8 of the 80 cells; non-invasiveness (byte-identical outputs) is recorded per measurement. On an 80-cell matrix, memset-based erasure loss is attributed to Dead Store Elimination, while assert-based checks and conditional-compilation guards vanish before any optimization runs; symbol-name search is shown invalid as a loss oracle, and an independent verifier that does not import the generator's canonicaliser checks the evidence.

Keywords: compiler optimization, security property loss, dead store elimination, stage attribution, independent verification

1. はじめに

ソースコードに書かれたセキュリティ上の判断は，そのまま実行される保証がない．秘密を消去する処理，権限のない要求を拒否する分岐，リリース構成でのみ有効になる検査も，観測可能な副作用を持たなければ規格の許す範囲で除去してよい．結果として，レビュー・静的解析・テストを通過した防御が利用者のバイナリには存在しない状態が生じる．

この現象自体は既知である（3 章）．本稿が扱うのは，どの工程で消えたかという一段先の問いである．工程により対処が変わるためである．前処理で消えた防御はビルド構成で復元でき，最適化パスが除去した防御は当該パスの抑止か除去され得ない形への書き直しで対処できる．しかし，バックエンドで消えた防御は中間表現の調査では捉えられず，工程を特定できなければ対処は根拠を欠く．

工程の特定には固有の困難がある．観測手段自体がコンパイラの内部表現に依存するため，単一測定から「このパスが落とした」とは断定できない．そこで本稿は，これを相補的な三観測チャネル（以下，三法）の一致で扱う．ただし，三者は同一のコンパイラ・中間表現・パイプラインを共有するため，一致は統計的独立の保証ではなく，観測チャネル間の照合である．

本稿の貢献は三点である．

（1）観測点の系列化．起動時のコマンドライン，構文木，最適化前の中間表現，各パス直後の中間表現，目的ファイ

* 所属：近畿大学 工学部（Faculty of Engineering, Kindai University）
連絡先：kondo.yuta.02@gmail.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ル，リンク後のイメージ，成果物の各点で同一の性質を同一の規約で数え，消失を局在化する．

（2）三法一致による工程帰属． -print-after-all によるパス直後の観測，-opt-bisect-limit によるパス予算の二分，そしてコンパイラを改変しない out-of-tree なパスプラグインの三つを実行し，同じパスを指したセルだけを帰属済みとして数える．なお，一致しなかったセルは一致しなかったと報告する．

（3）生成器の正準化器を用いない独立検証器． 測定結果は封緘され，生成側の正準化器を用いない検証器が別プロセスとして受理・拒否を判定する．ただし，検証器が受理したことと，測定が正しいことは別の主張であり，本稿はその区別も明示する（7 章）．

本稿の測定は合成 fixture 上で行われており，実在プロジェクトへの適用は行っていない．この制約と，性質語彙のうち実装が追いついていない部分は，7 章に列挙する．

2. 背景と脅威モデル

2.1 対象とする現象

問題の中心は，ソースの意味論とセキュリティの意図の不一致である．C/C++ の規格がコンパイラに要求するのは観測可能な振る舞い（揮発性アクセス・入出力・終了）の保存であり，「秘密がメモリに残らないこと」はそこに含まれない．寿命の尽きるバッファへのゼロ書き込みは規格上除去してよく，未定義動作を起こす経路は「起こらない」と仮定してよいので，null 参照後の null 検査は恒真として畳んでよい．どちらも仕様どおりの挙動であり，CWE-14 および CERT C MSC06-C として文書化された公知の現象である[2][3]．

この挙動が仕様どおりである点が対処を難しくする．バグであれば報告と修正で解消できるが，仕様どおりの最適化には修正すべき対象がなく，消失を測定して把握し，ソースの書き方を変える以外に対処はない．

2.2 脅威モデル

表 1 に本手法が扱う脅威と扱いを示す．

想定する攻撃者は，ソースに悪意ある変更を加えるのではなく，ソースに書かれた防御が成果物に到達しないという事実を利用する．前者はコードレビューと差分検査の問題だが，後者はレビューでは原理的に検出できない．レビュー対象はソースであって成果物ではないからである．

表 1　本手法が扱う脅威と扱い

Table 1　Threats and how this work addresses them.

脅威	内容	本稿での扱い
T1 性質消失	最適化・条件付きコンパイルで防御が消える	対象（中心）．6 点の観測と工程帰属
T2 構成ドリフト	未承認のフラグ・マクロ・最適化水準	対象．水準×緩和フラグの状態行列
T3 プラグイン注入	未承認のコンパイラプラグイン	非対象（明記）
T4 非ソース由来コード	ソースで説明できない関数・呼出し等	非対象（明記）
T5 リンク入力注入	未承認のオブジェクト・アーカイブ	非対象（明記）
T6 成果物改変	リンク後の書き換え	限定対応：artifact digest 照合
T7 証拠偽造	偽の証拠束	限定対応：署名＋独立検証（鍵保持者は範囲外）
T8 ツールチェーン汚染	改ざんされたコンパイラ／リンカ	非対象（digest の記録のみ．正当性は前提）
T9 不一致成果物	検証したものと違う配布物	限定対応：artifact ⇔ evidence 照合

信頼の基点として次を仮定する．コンパイラは改変されていない(本手法は改変されたコンパイラを検出できない)．測定環境は攻撃者の制御下にない（敵対的な環境では測定結果は偽造され得る）．署名鍵は測定対象の外にある（鍵の保持者は任意の内容を封緘できるため，封緘は発行主体を固定するのみで，内容の真正性は固定しない）．

2.3 非対象

設計時点で対象外と宣言するものが二つある．第一に，Trusting Trust の完全な解決である．コンパイラ自身が敵対的な場合，観測はそのコンパイラを経由するため信用できない．そのため，本手法は「正直なコンパイラが仕様に従って意図せず防御を落とす」ことを対象とする．第二に，全称的な説明閉包である．「セキュリティに関係する要素」の集合を計算可能に定義していない以上,「すべて説明した」とは主張できない．主張できるのは，宣言した性質・観測点・数え方の結果だけである．

3. 先行研究との関係

3.1 秘密消去の消失と工程帰属

秘密消去（scrubbing）がデッドストア除去で取り除かれる現象は，Yang らが体系的に扱い，実プロジェクト群に検出器を適用した[1]．本稿はこれを比較の基準として扱う．比較は主張・手法・範囲の対比であり，同研究の実験を本稿の環境で再実行した数値比較ではない．

差分 1 工程帰属． 同研究の検出器は Clang 3.9.0 のデッドストア除去 Pass を改造して実装され，消された消去を位置情報つきで報告する．「どこが消えたか」は同研究で既に得られている．本稿が加えるのは，どの工程で失われたかの帰属である．単一測定でパス名を特定すること自体は既存の機構でも可能であり（3.3 節），本稿の主張は三法が同じパスを指したセルだけを帰属済みと数える判定規約にある．また，同研究が改造版コンパイラを要するのに対し，本稿は無改造の配布版ツールチェーン上で成立する（5 章）．

差分 2 性質語彙の一般化． 同研究の対象は秘密消去に限定

© 2026 Information Processing Society of Japan

される．本稿は性質を語彙として宣言可能にし，消去以外の性質（失敗時に閉じる分岐，認可検査，構成フラグ，成果物に現れてはならない要素）を同じ枠組みで扱えるようにする．ただし，語彙として宣言できることと，それを測る抽出器が存在することは別である．実装が追いついていない範囲は 7 章に実測値とともに列挙する．ここで,「実装した」とは主張しない.

3.2 最適化による防御消失の広い実証

防御が正しさを保ったまま最適化で失われる現象の指摘は本稿が最初ではない．D’Silva らはこの correctness-security gap を定式化した[11]. STACK は, 未定義動作に依存して最適化で消えうるコードをソース解析で発見し，実システムから多数の事例を報告した[12]．ソース解析で観測できる消失類型が存在することは本稿の測定と矛盾せず, 前段の検出として相補する. Geimer らは定数時間実装について GCC/LLVM の漏えい原因パスを分析し，フラグによる選択的パス無効化という緩和まで示した[13]．性質を witness・注釈として最適化越しに運ぶ提案[14][15]，消去等をコンパイラ拡張で守る提案[16]，コンパイラ起因バグや Pass 起源の実証[17][18]も隣接するが，いずれも拡張を前提とするか工程帰属と証拠化を扱わない．対象性質を定数時間や未定義動作依存に限定せず，実ビルドにおける第一喪失点の観測・帰属・証拠化・独立検証を一つの運用に閉じる点が本稿の位置である（付録 C 表 C.5）.

3.3 機能的な変換検証との関係

Alive2 に代表される変換検証は，最適化が意味を保存するかを検証する[4]．変換検証が問うのは「この変換は規格上正しいか」, 本稿が問うのは「規格上正しい変換の結果として意図した性質が失われたか，どの工程でか」である．規格上正しいまま防御が消えるのが問題の本体であり，両者は同じ現象の異なる側面を扱い，優劣は論じない.

3.4 既存の機構と観測の限界

-opt-bisect-limit，-print-after-all，パスプラグイン機構は LLVM の公式機構であり[5][6], bugpoint や llvm-reduce もある[7]．本稿はこれらを部品として用い，機構の新規性は主張しない．なお，本測定環境（LLVM 18）では，最適化レポート機構（-fsave-optimization-record・-Rpass）はデッドストア除去による memset 除去を報告しなかった．削除の事実を公式レポートから得ることは現状期待できず，外部観測が必要である.

証拠の生成側と検証側の分離は，in-toto・SLSA のビルダ・検証器分離[8][9]，Proof-Carrying Code の生成器・検査器分離[10]と同型であり，設計の正しさの根拠として引く（新規性としては主張しない）.

4. 設計

4.1 観測点の系列

帰属先の各段階（前処理・AST・最適化 Pass・成果物）は，図 1 に示す形に精密化する．設計上の系列は 7 点であり，そこから定義した測定用の観測点は 6 点（preprocess / ast / ir-pre / ir-post / asm / artifact），実際に観測が得られた点はコンパイラにより異なる（clang-18 で 6 点中 5 点，gcc-13 で 2 点．6 章）.



図 1　観測点の系列（設計 7 点・測定 6 点）

Figure 1　The observation-point series: 7 designed, 6 measured.

観測点が系列をなす点が重要である．単一の観測点は性質の有無しか判定できないが，系列は値が変化した位置を特定できる．そこで本手法は，全観測点で同一の性質を同一の規約で数え，値が変わった最初の点を第一喪失点として記録する.

4.2 判定と状態の二層語彙

語彙は二層に分かれる．混同すると，観測できなかったことが「観測した結果，なかった」と誤って扱われる．各観測点で判定器が出力するのが判定である（表 2）.

判定の列から導かれるのが，そのセルの状態である（表 2）．観測消失率の分母に入るのは表 2 の PRESERVED から LOST までの 4 状態だけで，NOT_APPLICABLE と UNRESOLVED は分母から外して別に報告する．この分離を欠くと，観測できなかったセルを保存側にも消失側にも恣意的に算入できてしまう.

表 2　判定（観測点ごと）と状態（セルごと）の二層語彙

Table 2　Two-layer vocabulary of per-checkpoint verdicts and per-cell states.

値	意味
判定（観測点ごとに判定器が出す）	
PRESENT	効果が観測された
ABSENT	観測されなかった
NOT_APPLICABLE	問いの前提が失われた
INVALID_CONTROL	対照が効果を示さない（判定不能）
UNOBSERVED	本文が取れなかった（ABSENT と同一視しない）
状態（判定の列から導かれるセルの値）	
PRESERVED	最後まで観測された
TRANSFORMED_EQUIVALENT	表現は変わったが効果は等価
WEAKENED	残ってはいるが保証が落ちている
LOST	直前の観測点では観測され，ここで観測されなくなった
NOT_APPLICABLE	問いの前提が失われた（分母から外して別に報告）
UNRESOLVED	判定に必要な観測が取れなかった（分母から外して別に報告）

状態列は最後まで保持する．最初の消失で打ち切ると，一度消えて再び現れた性質（REINTRODUCED）を見落とし，

© 2026 Information Processing Society of Japan

実在する性質を「消えた」と報告する過検出になる（発動実績は 6 章）．INVALID_CONTROL と UNOBSERVED を独立に持つのも同じ理由で，到達しなかった判定経路と，判定器が判定不能となった構成を「検査したが何もなかった」と記録することは，網羅性に関する虚偽となる．なお，どちらも本測定で実際に発生している（7 章）．

4.3 性質語彙

性質は五つのクラスに分類される．must-survive（ソースに書かれた効果が成果物まで残ること．秘密消去，失敗時に閉じる分岐，認可検査），must-not-appear（現れてはならない呼出しや要素が現れないこと），must-originate-from（定義が承認された供給元から来ていること），must-be-configured（評価済みの構成でビルドされたこと），must-remain-unobservable（秘密が成果物から読み取れないこと）である．ただし，この五分類は語彙であって，実装のカバレッジではない．どのクラスにいくつの抽出器があり，どの観測点で測れているかは 7 章に実測値を付して示す．

4.4 オラクル規約

測定の妥当性は本節の規約に依存し，以下の四つを欠くと得られる数値は意味をなさない．

規約 1：消失をシンボル名の検索で判定しない．最適化水準が上がると，消去処理は呼び出しではなくインライン化されたゼロ書き込み命令として実現される．そのため，名前を検索する判定器は，この時点で削除不可能な対照までも「消えた」と報告する．判定は名前ではなくゼロ化命令の計数で行う．

規約 2：中間表現では呼び出し地点を数える．宣言行は，その呼び出しがすべて消えたあとも残る．宣言の存在を性質の存在と読むと，消失は観測されない．

規約 3：削除不可能な対照を必ず同居させる．各セルには，どの最適化水準でも除去できない対照を必ず同じ翻訳単位に置き，効果の計数が 0，対照が非 0 に分かれることを毎回示す．したがって，両方が 0 になったセルは，性質が消えた証拠ではなく，判定器が正しく機能していない証拠として扱う．なお，この規約は，報告する数の選び方も制約する．「消失を報告した行では対照が保たれていた」という記述は証拠にならない．判定器は対照が 0 になった時点で判定を INVALID_CONTROL に変更するため，対照が失われた消失行は定義上存在せず，故障時も同じ数が得られるからである．報告すべきは，判定器が判定不能となった枠の数である（6 章）．

規約 4：ガードは分岐が生きている間だけ数える．定数に畳まれた分岐は制御を行わない．畳まれた分岐に従属する拒否経路は，呼び出し文字列が残っていても防御として機能しておらず，本稿はこの時点を消失とみなす．この規約を適用しない場合の帰属のずれは数値では述べない．生存分岐で絞らずに数えた記録がなく，推定にならざるを得ないためである．一方，規約 1・2 については記録があり，6 章で数字を伴って述べる．

5. 実装

5.1 非侵襲性の二条件

本手法は次の二条件を満たす形で実装される．(i) コンパイラを改変しない．公式に提供されているフラグと，out-of-tree なパスプラグイン機構のみを用いる．改変版コンパイラを配布する必要がないため，測定対象と利用者のツールチェーンが一致する．(ii) 観測器の有無で生成物がバイト一致する．観測を有効にしたビルドと無効にしたビルドで，目的ファイルと実行ファイルのダイジェストが一致することを測定する（結果は 6 章）．

条件 (ii) には，観測器が一度も発火しないまま生成物が一致する危険がある．そこで，発火そのものを別に記録し，一致と発火が同時に成立していることを示す．バイト一致のみの報告では，観測を行わず変化がなかった場合と区別できないからである．なお，実装の名称は VibeGuard Compiler であるが，新たなコンパイラを実装したものではない．実体は公式フラグ・純観測パスプラグイン・それらを束ねるスクリプト群である．測定系を含む基盤全体の実装済みの範囲は付録 F に示す．

5.2 三法の実現機構

三つの帰属方法は，いずれも公式の機構のみで実現される[5][6]（付録 C 表 C.2）．第一に，パス直後の中間表現の観測である．全パスのダンプを取り，対象と対照を同じ規約で数える．関数単位のダンプはその関数しか含まないため，対照が別の単位にある場合は対照を含む直近のダンプを対照側の入力として与える．第二に，パス予算の二分である．パス数の上限を与えてビルドし，性質が失われる最小の予算を求める．予算 n で失われ n−1 で失われないなら，n 番目に実行されるパスの実行時点を第一消失として帰属する．-opt-bisect-limit が数えるのは opt-in されたスキップ可能パスのみであり，必須パスと解析パスは対象外である[6]．また，先行パスが削除の前提条件を作る場合があるため，この帰属は「宣言した観測述語が最初に偽になったパス実行」であって，根本原因の因果的特定ではない．この方法は第一の方法と内部表現を共有しないため，相補的な照合になる．第三に，out-of-tree パスプラグインである．コンパイラを改変せずに，パスの前後で状態を記録する．

三つが同じパスを指したセルを一致セルとして数え，一方しか帰属できなかったセルは別に数える．また，矛盾したセルと未解決のセルは，0 件であっても 0 件として報告する．これらを報告しなければ，一致率を分母の選び方で恣意的に高くできるためである．

5.3 成果物層とリンク後

成果物層では ELF の構造を直接読む．ツールの要約行ではなく構造フィールドを根拠にするため，判定がどのフ

© 2026 Information Processing Society of Japan

ィールドで決まったかを記録に残す．また，到達しない判定経路は UNOBSERVED として記録し，検査済みとは扱わない．

5.4 証拠と検証

各セルの測定結果は正準化され，封緘される（付録 C 図 C.1）．検証は生成側の正準化器を用いない別プロセスとして実行され，束の集合全体に判定を下す．判定は終了コードではなく判定文字列で行う．複数の拒否理由が同一終了コードに畳まれ，「なぜ拒否されたか」が失われるからである．なお，終了コードは併記し，食い違えばその食い違いを報告する．判定は VERIFIED_CLEAN ／ FINDINGS_PRESENT ／ VERIFICATION_INCOMPLETE ／ UNSUPPORTED／EVIDENCE_MISMATCH の 5 状態である．VERIFIED_CLEAN は束の形式・ハッシュ・内部整合性の検査に合格したこと（束整合性合格）を意味し，測定対象のセキュリティ的安全や LOST セルの不在を意味しない．封緘が保証しないことは 7 章に列挙する．また，署名はバイト列を鍵の保持者に結びつけるだけであり，測定であることを保証しない．

6. 評価

6.1 測定の設定

測定は 80 セル＝5 性質×2 コンパイラ（clang-18 18.1.3・gcc-13 13.3.0）×4 最適化水準（-O0〜-O3）×緩和フラグの有無，で構成する．第 4 の軸である緩和フラグは，性質別のビルドトグル，すなわち当該性質の可観測性を左右するビルド指定の有無（条件付きコンパイル指定を含む）として定義する．期待方向は性質ごとに逆になり得る（例：-DNDEBUG は assert を無効化する側の指定である．表 3）．また，第一喪失点が前処理の 16 セルは，コンパイラ最適化ではなくマクロ意味論・ビルド構成による消失である（6.2 節）．なお，環境は WSL2 上の Ubuntu 24.04 であり，ビルド時刻は SOURCE_DATE_EPOCH で固定して全結果に記録する．

fixture は表 3 に示す 5 本に，成果物層専用の notappear を加えた 6 本である．各 fixture は，どの水準でも除去できない削除不可能な対照を対象と同じ翻訳単位に同居させ，manifest で宣言する．また，秘密や検査結果は別翻訳単位の不透明な consumer に渡して実体化を強制し，測定対象そのものが退化して消えることを防ぐ．

表 3　fixture とソース上の防御・緩和フラグ

Table 3　Fixtures, their source-level defenses and mitigation flags.

fixture	ソース上の防御	緩和フラグ
authz	assert による認可検査	-DNDEBUG
configguard	#ifdef による防御	-DENABLE_AUTH
erasure	memset による秘密消去	-fno-builtin-memset
nullcheck	null 検査	-fno-delete-null-pointer-checks
signedovf	符号付き overflow 検査	-fwrapv

6.2 消失行列と第一喪失点

80 セルのうち保存は 42 セル，消失は 38 セルである．また，第一喪失点の分布は，前処理 16・最適化 Pass 8・バックエンド 1・段階を特定できずコンパイル過程全体に帰属したもの 13 である．一方，AST 層と成果物層を第一喪失点とするセルは観測されなかった．これは，AST 層を区別する機構（前処理出力と AST ダンプの別発行）は実在するが，そこを第一喪失点とする実例は本測定では現れなかったことを意味する．図 2 に全セルの状態と第一喪失点を示す（全 80 セルの一覧は付録 B 表 B.1）．

		clang-18				gcc-13			
	緩和	-O0	-O1	-O2	-O3	-O0	-O1	-O2	-O3
authz	なし	−	−	−	−	−	−	−	−
	あり	前	前	前	前	前	前	前	前
configguard	なし	前	前	前	前	前	前	前	前
	あり	−	−	−	−	−	−	−	−
erasure	なし	−	後	IR	IR	−	中	中	中
	あり	−	−	−	−	−	中	中	中
nullcheck	なし	−	IR	IR	IR	−	中	中	中
	あり	−	−	−	−	−	−	−	−
signedovf	なし	−	IR	IR	IR	中	中	中	中
	あり	−	−	−	−	−	−	−	−

網掛け＝消失（38 セル）

図 2　状態行列（80 セル）．行＝性質×緩和フラグ，列＝コンパイラ×最適化水準．−＝保存，網掛け＝消失．消失セルの記号は第一喪失点（前＝前処理，中＝コンパイル過程，IR＝IR Pass，後＝バックエンド（観測上））

Figure 2　State matrix for all 80 cells. A shaded cell is lost, labeled by its first-loss stage.

消失は -O1 以上の水準に限らない．-O0 でも 5 セルが失われる（前処理での削除 4，および gcc-13／-fwrapv なし／-O0 の signedovf 1 セル）．また，-O1・-O2・-O3 の消失セル集合は個数（各 11 セル）だけでなく集合として一致し，-O0 の消失集合（5 セル）はその真部分集合である（図 3）．

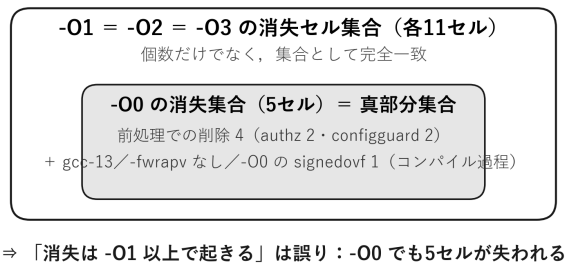


図 3　消失セル集合の入れ子関係

Figure 3　Nesting of the loss-cell sets across optimization levels.

© 2026 Information Processing Society of Japan

表 4　性質別の消失セル数と観測消失率

Table 4　Per-property loss counts and observed loss rates.

性質	消失	評価	観測消失率	三法一致	段階のみ
authz.assert	8	16	50%	0	8
configguard.ifdef	8	16	50%	0	8
erasure.wipe	9	16	56.3%	2	7
nullcheck.guard	6	16	37.5%	3	3
signedovf.guard	7	16	43.8%	3	4

性質別の観測消失率を表 4 に示す. 観測消失率は消失セル数を評価セル数で割った値であり，分母に入るのは判定が確定した 4 状態のみ，NOT_APPLICABLE と UNRESOLVED は分母から外して別に報告する（本測定ではいずれも 0 件．6.3 節）．authz 50%・configguard 50%・erasure 56.3%・nullcheck 37.5%・signedovf 43.8% である．ただし，この値は著者が固定した評価行列の中の記述統計であり，セルは同一 fixture の構成違いで独立標本ではないため，一般のプロジェクトにおける発生確率の推定値としては解釈できない.

同一の性質でも消える工程は一様でない. 例えば, erasure（memset 消去）の消失 9 セルの内訳は，clang-18 の -O2／-O3 では三法一致でデッドストア除去（DSEPass）に帰属した一方，同じ clang-18 でも -O1 では中間表現に消去が残り，アセンブリに現れない（バックエンド観測．この表現の限界は 7 章(4)）. これに対し, gcc-13 の 6 セルは Pass 粒度で観測できない区間（コンパイル過程全体）で消えた．また，assert 認可と #ifdef 防御が前処理で消える 16 セルには例外がない．これらは最適化が一つも実行される前に既に消えており, Pass 単位の帰属では原理的に説明できない.

6.3 三法一致による帰属

表 5　三法一致による工程帰属の内訳

Table 5　Attribution by three-method agreement.

項目	件数	備考
行列セル	80	5 性質×2 コンパイラ×4 水準×2 緩和
消失セル	38	保存 42
三法一致で帰属確定	8	DSEPass 2・EarlyCSEPass 6．clang-18 のみ・3/5 性質
段階のみ特定（stage-only）	30	前処理 16・バックエンド 1・コンパイル過程全体 13
帰属の矛盾	0	ゼロでもゼロと報告する
未解決（trace 層）	0	成果物層の判定不能 16 枠は表 7
二法のみ一致（「暫定」表記）	0	規約は用意・発動なし
再導入（REINTRODUCED）	0	機構は用意・発動なし

三法が同一 Pass を指した場合に限り帰属確定とした．この基準を満たしたのは 80 セル中 8 セル（DSEPass 2・EarlyCSEPass 6）であり，いずれも clang-18，5 性質中 3 性質である（表 5）．また，段階のみ特定できたセルは 30，帰属の矛盾と未解決（trace 層）は 0 である（0 件であることも報告する）．規約上「暫定」と表記される二法以下一致セルの発動は 0 件，REINTRODUCED マーカーの発動も 0 件であり, 機構の用意と発動は別であるため, 発動しなかったことを報告する．さらに，三法の内部整合も確認した．各方法の対照が消失しないこと 8/8，二分探索の遷移点前後の単調性 8/8（付録 C 表 C.7），二分とダンプ走査の相互一致 8/8，プラグインとダンプ走査の相互一致 8/8 である．各チャネル単独の帰属との全数一致は付録 E.1 に示す. また, 帰属 Pass を抑止する介入では, 防御が最終アセンブリまで復活した性質, 中間表現でのみ復活した性質, 復活しなかった性質に分かれた．帰属 Pass は，抑止すれば防御が戻る唯一の原因ではない（付録 E.2）.

6.4 オラクルの対照実験

規約 1・2 の実証として，erasure ファミリ 8 構成で名前検索とゼロ化命令オラクルを並走させた（表 6）. 名前検索は 6 構成で削除不可能な対照までも「消えた」と報告した. 一方, ゼロ化命令オラクルが同じ誤りを犯した構成は 0 である．さらに帰属先も変化する．具体的には，対象関数だけを含む翻訳単位では, 名前検索は GlobalOptPass（パス予算 59）を，呼出し地点オラクルは DSEPass（予算 50）を指し, 名前検索に従うと 9 段後方のパスへ誤って帰属される．ただし，ゼロ化命令の計数自体にも既知の欠陥がある（7 章（16））.

表 6　オラクルの対照実験（erasure ファミリ 8 構成）

Table 6　Control experiment on loss oracles (8 configurations).

項目	名前検索	本稿のオラクル
対照まで「消えた」と報告	8 構成中 6	8 構成中 0（ゼロ化命令）
特定した Pass	GlobalOptPass（予算 59）	DSEPass（予算 50）（呼出し地点）

また, 対照の非退化も報告する. 中間表現までの trace 層の観測枠（80 セル×5 観測点＝400 枠）のうち，対照の読み取りが得られた 280 枠はすべて非 0 で，0 だった枠はない．一方，残る 120 枠は gcc-13 側の中間表現層で対象側の読み取りが得られなかったものであり，観測不能であって陰性ではない．判定器が判定不能となった枠（INVALID_CONTROL）は trace 層では 0 だが, 成果物層では 16 枠が該当し（6.5 節），「対照はどこでも保たれた」とは言えない.

6.5 成果物層

表 7　成果物層の判定内訳（clang-18 側）

Table 7　Artifact-layer verdicts for clang-18.

判定	件数	備考
排他的判定の対象	40	5 fixture×4 水準×2 緩和
PRESENT	13	防御が成果物に残存
ABSENT	11	成果物に不在
INVALID_CONTROL	16	nullcheck・signedovf の全セル＝オラクル判定不能
must-not-appear 残存検出	16/16	専用 fixture 8 セル・16 読取（排他的判定と別枠）

成果物層の判定は clang-18 側で行った（表 7）．排他的判定の対象は 40 セル（5 fixture×4 水準×2 緩和）であり，PRESENT 13・ABSENT 11 に対し，nullcheck と signedovf は全セルで対照が成果物層の観測に現れず，判定は

© 2026 Information Processing Society of Japan

INVALID_CONTROL である（7 章（12））．これとは別に，must-not-appear（現れてはならない要素）の専用 fixture（8 セル・16 読取）は全読取で残存を検出した．なお，成果物層の観測は静的観測であり，実行時にメモリへ秘密が残留するか否かはここでは主張しない．

6.6 非侵襲性と決定性

観測器の有無による生成物の比較は 80 セル全てで行い，目的ファイル 80/80・実行ファイル 80/80 が一致した．同時に，観測器が発火した記録が 80 セル全てに存在し，観測を無効にした側のビルドに観測器出力が存在しないことも確認した．比較が変化を検出できること自体の陰性対照として，-opt-bisect-limit=40 を与えたビルドでは目的ファイルが実際に変化した．ただし，clang-18 側 40 セルは純観測プラグインの有無による比較，gcc-13 側 40 セルは -fdump 系ダンプ有効化の有無による比較であり，別チャネル・別主張である（この比較チャネルは段階帰属に用いていない）．また，決定性はセル単位で確認した．同一構成の再ビルドは 80/80 セルでダイジェスト一致，二度ビルドのプローブは 2/2 で一致である．決定性の主張はこの範囲に限る．

6.7 証拠束と独立検証

clang-18 側 40 セルの証拠を正準化して封緘した（署名は ed25519，鍵は束の外）．生成側の正準化器を用いない検証器は，束あたり 15 検査を実行し，全 40 束を VERIFIED_CLEAN（束整合性合格）と判定した．較正失敗（破損させた束を受理する誤り）は 0（付録 C 表 C.6），終了コードと判定文字列の食い違いは 0，実行 manifest との整合も確認した．一方で，検証時点で束の署名が覆っていない結果ファイルが 17 件ある（7 章．署名サイドカーによる事後被覆は付録 E.4）．また，測定枠組みの妥当性判読は 5 fixture・引用 32 行に対して行ったが，その等級は prepared-only である．判読者は LLM エージェントであり，人手による追認を経ていない．本稿はこれを目視による確認とはみなさない．

6.8 ソース側検出への閉ループ

本稿が「消える」と指摘する分類のうちソースコード段階で検出できる 2 件は，検出ルール（VG-AUTH-008・VG-MEM-006）として実装・出荷済みであり（研究倫理の節），閉ループの記録は 2 件とも，ソースを修正した後の再測定で当該性質が成果物まで生存したことを示す RESOLVED_AND_SURVIVED である．これは，証拠から独立検証を経てソース側検査へ戻る経路が，一方向の指摘で終わらないことの最小の実証である．

7. 限界と妥当性への脅威

本節では本稿の限界を述べる．以下は全て実測に基づく．

（0）観測器そのものの限界（全数値が依存する前提）．観測器には，エラーを出さずに観測を行わない失敗形態が 3 つある．うち 1 つ（観測対象関数名の指定誤り）は，終了コード 0・エラー出力なし・記録は非空・陽性対照は PRESENT のまま対象の行だけが存在しないという形をとり，「陽性対照を同じ記録に併記する」不変条件では検出できない．80 セルがこの形態の影響下にないことは事後の全セル走査で確認した（該当する失敗形態 0 件．対照 0 の 40 セルは gcc-13 側の UNOBSERVED）．ただし，この検査は測定時には存在せず，自動で測定を止める機構もない．「観測器は健全である」とは主張しない．

（1）合成 fixture のみ．本稿の測定は著者が作成した最小の合成プログラム上で行われており，実在プロジェクトに対する適用は行っていない．

（2）三法一致は 80 セル中 8 セル，clang-18 のみ．三法一致を全セルの標準手続きとしては提示しない．2 法以下しか揃わないセルは「暫定」と表記する規約を置くが，本測定での発動は 0 件である．

（3）性質語彙 5 クラスのうち実装があるのは 3 クラス．性質カタログ 24 プロパティ中 implemented は 7（must-survive 3・must-not-appear 1・must-be-configured 3）．partial 1・candidate 2・unimplemented 14（付録 C 表 C.1）．implemented が 0 のクラスは，must-originate-from（由来分類の機構は在るが性質としては未宣言）と must-remain-unobservable（抽出器・観測点・測定のいずれもない）である．本実験で測った性質の数（5）はカタログの規模（24）とは別の数である．

（4）-O1 の消失 1 セルを中間表現の Pass に帰属できない．当該セルの ir-post と asm は同一起動の 2 地点ではなく別々の 2 回のコンパイルであり，-emit-llvm はパスパイプライン自体を変える（同一セルでも総パス数が一致しない）．観測が支えるのは「中間表現には在り，アセンブリにはない」までで，「バックエンドで消えた」は推論を含むため，この 1 セルは「バックエンド（観測上）」と表記した．

（5）成果物層の一部の判定経路は到達不能．.init_array 解決の 4 分岐のうち R_X86_64_IRELATIVE 経路は本測定環境のどの成果物でも発火せず（libc.so.6 にも 0 件），UNOBSERVED と記録した．「検査済み」とは扱わない．

（6）改竄検出の範囲．証拠は，artifact と evidence の不整合および evidence 内部の矛盾を検出でき，鍵を持たない相手による事後改変を検出する．束全体を整合的に再生成できる相手（署名鍵の保持者を含む）は検出範囲外である．

（7）リンカ依存．.init_array の R_X86_64_RELATIVE 経路は lld のときだけ通る．一方，bfd はスロットを事前充填するため別経路になる．リンカを固定しない限り「全経路を検査した」とは主張できない．

（8）実施しなかった評価．人間の判読者による評価は，倫理手続きが本稿の期限までに完了しなかったため実施していない（UNSUPPORTED と記録）．

（9）履歴に残った公開物．過去に公開した計測用データの

© 2026 Information Processing Society of Japan

一部がリポジトリ履歴に残っており，遡及的な除去は配布物の利用者に不整合を生じさせるため行わない．工程帰属の段階語彙は追跡ファイルに露出しているが，証拠プロトコルの変更になるため数値凍結前には変更していない．

（10）公開 issue の本文改訂は編集履歴に残る．露出は減ったが 0 にはなっていない．改訂と露出の消失とは同一視しない．

（11）「-O1 以上でしか消えない」は誤りである．-O0 でも 5 セルが失われた（6.2 節）．うち gcc-13／-O0 の signedovf 1 セルは前処理ではなくコンパイル過程での消失である．ただし，gcc セルは 6 観測点中 2 点のみでの観測であり，「最適化が落とした」とまでは言えない．

（12）対照が 0 になる層が実在する．成果物層の 16 枠は INVALID_CONTROL であり（表 7），「対照はどこでも保たれた」とは言えない．

（13）並列 ThinLTO では観測器がリンカを異常終了させる．プラグインを適用した lld の ThinLTO リンクは並列実行でメモリ破壊により異常終了し，挙動も非決定的だった（--thinlto-jobs=1 では安定．試作版観測器での観測）．観測器はスレッド安全でない．

（14）LTO バックエンドには到達できるが測定できない．full LTO で観測器はリンク時バックエンドに到達しモジュールを特定できるが，対照の除去を妨げる別翻訳単位を LTO が併合するため，対照の計数が全 fixture で 1→0 に落ちる．規約 3 に照らしこれは判定器が判定不能となる構成である．

（15）エラーを報告しない失敗．LTO リンク行に -fpass-plugin= と誤記すると，エラーにならず exit 0 のままログが空になる（正しくは --load-pass-plugin=）．「ログが空」と「削除がなかった」は区別できないため，LTO を測るには誤指定の陰性対照の同居が必要である．

（16）ゼロ化計数の対象束縛の欠如．ir 側のゼロ化計数は，ゼロ定数ストアの基底が alloca か引数ポインタであることのみを要求し，対象バッファへの束縛（alias・バイト範囲・系譜）を検査しない．較正では，対象の wipe が除去された翻訳単位で無関係なゼロ初期化が計数を保ち，対照が保たれたまま PRESENT と誤読される構成が実測された．また，asm 側の計数も同種の欠陥を共有するため，チャネル間の照合ではこの誤りを検出できない．既知の欠陥の中で唯一 PRESENT 側へ誤るものである．本測定の erasure fixture の対象翻訳単位には無関係なゼロストアがないことを確認しているが，一般のコードに適用するには対象束縛の実装が先行する必要がある．対象束縛を実装した試作オラクルの最小実証を付録 E.3 に示す．

8. おわりに

本稿では，先行研究が示した工程特定を無改造の配布版 LLVM 18 の上で再構成し，独立検証可能な証拠として出力する手法を示した．改造版コンパイラによる先行研究との差分は，無改造の配布版ツールチェーンで成立すること，対象性質が秘密消去に限定されないこと，消失段階が最適化 Pass に限定されないこと，証拠の生成から独立検証・ソース側検出への閉ループを持つことの 4 点である．また，実装済みの範囲と限界は実測に基づいて区別して報告した（7 章）．今後の課題は，AST 層・成果物層を第一喪失点とする実例の観測，gcc の Pass 粒度での帰属，LTO 構成の測定可能性，未実装の性質クラスの抽出器，実在プロジェクトへの適用である．

参考文献

[1] Yang, Z., Johannesmeyer, B., Olesen, A. T., Lerner, S. and Levchenko, K.: Dead Store Elimination (Still) Considered Harmful, Proc. 26th USENIX Security Symposium (2017).

[2] MITRE: CWE-14: Compiler Removal of Code to Clear Buffers. https://cwe.mitre.org/data/definitions/14.html (参照 2026-08-10).

[3] Software Engineering Institute: SEI CERT C Coding Standard, MSC06-C. Beware of compiler optimizations. https://wiki.sei.cmu.edu/confluence/display/c/ (参照 2026-08-10).

[4] Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z. and Regehr, J.: Alive2: Bounded Translation Validation for LLVM, Proc. PLDI (2021).

[5] LLVM Project: Writing an LLVM Pass (New Pass Manager) ／ Pass Instrumentation. https://llvm.org/docs/WritingAnLLVMNewPMPass.html (参照 2026-08-16).

[6] LLVM Project: Bisecting miscompilations with -opt-bisect-limit. https://llvm.org/docs/OptBisect.html (参照 2026-08-16).

[7] LLVM Project: bugpoint ／ llvm-reduce. https://llvm.org/docs/CommandGuide/index.html (参照 2026-08-16).

[8] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R. and Cappos, J.: in-toto: Providing farm-to-table guarantees for bits and bytes, Proc. 28th USENIX Security Symposium (2019).

[9] SLSA: Supply-chain Levels for Software Artifacts v1.0. https://slsa.dev/spec/v1.0/ (参照 2026-08-16).

[10] Necula, G. C.: Proof-Carrying Code, Proc. POPL (1997).

[11] D'Silva, V., Payer, M. and Song, D.: The Correctness-Security Gap in Compiler Optimization, Proc. IEEE SPW (LangSec) (2015).

[12] Wang, X., Zeldovich, N., Kaashoek, M. F. and Solar-Lezama, A.: Towards Optimization-Safe Systems: Analyzing the Impact of Undefined Behavior, Proc. SOSP (2013).

[13] Geimer, A. and Maurice, C.: Fun with Flags: How Compilers Break and Fix Constant-Time Code, Proc. DIMVA (2026).

[14] Namjoshi, K. S. and Tabajara, L. M.: Witnessing Secure Compilation, Proc. VMCAI (2020).

[15] Vu, S. T., Heydemann, K., de Grandmaison, A. and Cohen, A.: Secure Delivery of Program Properties through Optimizing Compilation, Proc. CC (2020).

[16] Simon, L., Chisnall, D. and Anderson, R.: What You Get is What You C: Controlling Side Effects in Mainstream C Compilers, Proc. IEEE EuroS&P (2018).

[17] Xu, J., Lu, K., Du, Z., Ding, Z., Li, L., Wu, Q., Payer, M. and Mao, B.: Silent Bugs Matter: A Study of Compiler-Introduced Security Bugs, Proc. 32nd USENIX Security Symposium (2023).

[18] Du, Y., Alrawi, O., Snow, K. Z., Antonakakis, M. and Monrose, F.: Improving Security Tasks Using Compiler Provenance Information Recovered at the Binary-Level, Proc. ACM CCS (2023).

© 2026 Information Processing Society of Japan

研究倫理

本研究は，実在するコンパイラでソース上の防御が成果物から消えることの公表と，それを再現する最小コード例の掲載を含む点で，倫理的な問題に関係する．関係する論点と対処を（1）〜（3）に示す．いずれも適切に対処されていると判断する．

（1）実在するコンパイラで防御が消えることの公表．対象は公開済みの安定版コンパイラであり，示す消失は最適化の仕様どおりの挙動であって未知の脆弱性ではない．現象自体は 2017 年以降公知であり，CWE-14 および CERT C MSC06-C として文書化されている[2][3]．修正されるべき欠陥ではなく測るべき現象として報告するものであり，新規の悪用手段を与えないため，ベンダへの事前通知は不要と判断した．

（2）指摘に先立つ修正と出荷．本稿が「消える」と指摘する分類のうちソースコード段階で検出できるものは，指摘に先立ち，著者が開発するソースコード検査ツールの検出ルールとして実装し出荷済みである（VG-AUTH-008：NDEBUG 下で消える assert 認可検査，severity high／VG-MEM-006：除去され得る秘密消去）．両ルールは 0.3.5（2026-08-06）で出荷され，本稿執筆時点の現行版（0.3.6）にも含まれる．本研究は，問題を発見してそのまま報告したのではなく，検出可能なものを修正して出荷した上で残る現象を報告する順序を採った．

（3）最小コード例の転用可能性．掲載する例は合成 fixture であり，memset／assert／#ifdef という教科書的な既知パターンのみからなる．実在システムの秘密も攻撃経路も含まず，消失を観測するための最小形であって悪用のためのレシピではない．

付録

付録 A　再現手順

測定環境は WSL2 上の Ubuntu 24.04，コンパイラは clang-18（18.1.3）と gcc-13（13.3.0）である．測定は 20 ステージのパイプラインとして一括実行され，途中生成物と結果 JSON はすべてステージが書き出す．ビルド時刻は SOURCE_DATE_EPOCH で固定し，全結果 JSON に生成時刻として記録する．ツールチェーンの同定はパッケージ版数とコンパイラバイナリの sha256 で行う．正準ダイジェスト（toolchain.json・コンパイラ等 10 バイナリ）と，解析器・検証器スクリプト一式（35 ファイル）のソート済み sha256 列挙のダイジェストは次のとおりである．

```
toolchain
26806b03d68c449c9618f3da3420af4a33fa176cdd9a8d7566e594c9f1051d19
scripts
2cafd24236190be9f537dacd8776f064917b736c2454dc6942b64f31cb7505c3
```

本稿の数値・図表・証拠束は 2026-08-18 の正準ラン（20 ステージ）の出力と突合して確定した．

再現の手順は次のとおりである．(1) パイプラインの一括実行（結果ディレクトリは削除・再生成される）．(2) 決定性検査の明示実行（対照を含む）．(3) 数値マクロの再生成と，原稿側の未定義参照検査．本文の数値は全て生成されたマクロ経由であり，手動で記載した数値は引用年と定型数のみである．

検証器は生成側の正準化器を用いない別プロセスであり，判定は終了コードではなく判定文字列で読む．5 状態の読み方を表 A.1 に示す．検証器の受理は測定の正しさの証明ではない（7 章（0）（6））．

実装（ドライバ・観測器・証拠束の生成器と検証器）は https://github.com/YUTAKONDO1205/VibeGuard の commit 33c2ac5 にある．本稿の 80 セルを生成した 20 ステージのパイプラインは同リポジトリに含まない．検証経路は生成側の正準化器を import せず，正準テキストを規約から再導出する．ただし計数と経路検査のユーティリティは生成側と共有する．

表 A.1　検証器の判定 5 状態

Table A.1　The five verdicts of the independent verifier.

判定	意味
VERIFIED_CLEAN	束整合性合格・内部矛盾なし
FINDINGS_PRESENT	測定は受理・指摘あり
VERIFICATION_INCOMPLETE	検査を完了できず
UNSUPPORTED	対象外
EVIDENCE_MISMATCH	artifact と evidence の不整合または evidence 内部の矛盾

付録 B　全 80 セルの状態と帰属

表 B.1 に全 80 セルの状態（P＝PRESERVED／L＝LOST）・第一喪失点・帰属 Pass（宣言述語が最初に偽になった Pass 実行であり，根本原因を意味しない）・帰属の一致水準（三法＝三法一致／段階＝段階のみ）を示す．緩和列は緩和フラグの有無である．

表 B.1　全 80 セルの状態・第一喪失点・帰属

（次頁に続く）

Table B.1　State, first-loss stage and attribution for all 80 cells.

性質	コンパイラ	水準	緩和	状態	第一喪失点	帰属 Pass	一致
authz	clang	-O0	なし	P	—	—	—
authz	clang	-O0	あり	L	前処理	—	段階
authz	clang	-O1	なし	P	—	—	—
authz	clang	-O1	あり	L	前処理	—	段階
authz	clang	-O2	なし	P	—	—	—
authz	clang	-O2	あり	L	前処理	—	段階
authz	clang	-O3	なし	P	—	—	—
authz	clang	-O3	あり	L	前処理	—	段階
authz	gcc	-O0	なし	P	—	—	—
authz	gcc	-O0	あり	L	前処理	—	段階
authz	gcc	-O1	なし	P	—	—	—
authz	gcc	-O1	あり	L	前処理	—	段階
authz	gcc	-O2	なし	P	—	—	—

© 2026 Information Processing Society of Japan

性質	コンパイラ	水準	緩和	状態	第一喪失点	帰属 Pass	一致
authz	gcc	-O2	あり	L	前処理	—	段階
authz	gcc	-O3	なし	P	—	—	—
authz	gcc	-O3	あり	L	前処理	—	段階
configguard	clang	-O0	なし	L	前処理	—	段階
configguard	clang	-O0	あり	P	—	—	—
configguard	clang	-O1	なし	L	前処理	—	段階
configguard	clang	-O1	あり	P	—	—	—
configguard	clang	-O2	なし	L	前処理	—	段階
configguard	clang	-O2	あり	P	—	—	—
configguard	clang	-O3	なし	L	前処理	—	段階
configguard	clang	-O3	あり	P	—	—	—
configguard	gcc	-O0	なし	L	前処理	—	段階
configguard	gcc	-O0	あり	P	—	—	—
configguard	gcc	-O1	なし	L	前処理	—	段階
configguard	gcc	-O1	あり	P	—	—	—
configguard	gcc	-O2	なし	L	前処理	—	段階
configguard	gcc	-O2	あり	P	—	—	—
configguard	gcc	-O3	なし	L	前処理	—	段階
configguard	gcc	-O3	あり	P	—	—	—
erasure	clang	-O0	なし	P	—	—	—
erasure	clang	-O0	あり	P	—	—	—
erasure	clang	-O1	なし	L	バックエンド	—	段階
erasure	clang	-O1	あり	P	—	—	—
erasure	clang	-O2	なし	L	IR Pass	DSEPass	三法
erasure	clang	-O2	あり	P	—	—	—
erasure	clang	-O3	なし	L	IR Pass	DSEPass	三法
erasure	clang	-O3	あり	P	—	—	—
erasure	gcc	-O0	なし	P	—	—	—
erasure	gcc	-O0	あり	P	—	—	—
erasure	gcc	-O1	なし	L	コンパイル	—	段階
erasure	gcc	-O1	あり	L	コンパイル	—	段階
erasure	gcc	-O2	なし	L	コンパイル	—	段階
erasure	gcc	-O2	あり	L	コンパイル	—	段階
erasure	gcc	-O3	なし	L	コンパイル	—	段階
erasure	gcc	-O3	あり	L	コンパイル	—	段階
nullcheck	clang	-O0	なし	P	—	—	—
nullcheck	clang	-O0	あり	P	—	—	—
nullcheck	clang	-O1	なし	L	IR Pass	EarlyCSEPass	三法
nullcheck	clang	-O1	あり	P	—	—	—
nullcheck	clang	-O2	なし	L	IR Pass	EarlyCSEPass	三法
nullcheck	clang	-O2	あり	P	—	—	—
nullcheck	clang	-O3	なし	L	IR Pass	EarlyCSEPass	三法
nullcheck	clang	-O3	あり	P	—	—	—
nullcheck	gcc	-O0	なし	P	—	—	—
nullcheck	gcc	-O0	あり	P	—	—	—
nullcheck	gcc	-O1	なし	L	コンパイル	—	段階
nullcheck	gcc	-O1	あり	P	—	—	—
nullcheck	gcc	-O2	なし	L	コンパイル	—	段階
nullcheck	gcc	-O2	あり	P	—	—	—
nullcheck	gcc	-O3	なし	L	コンパイル	—	段階
nullcheck	gcc	-O3	あり	P	—	—	—
signedovf	clang	-O0	なし	P	—	—	—
signedovf	clang	-O0	あり	P	—	—	—
signedovf	clang	-O1	なし	L	IR Pass	EarlyCSEPass	三法
signedovf	clang	-O1	あり	P	—	—	—
signedovf	clang	-O2	なし	L	IR Pass	EarlyCSEPass	三法
signedovf	clang	-O2	あり	P	—	—	—
signedovf	clang	-O3	なし	L	IR Pass	EarlyCSEPass	三法
signedovf	clang	-O3	あり	P	—	—	—
signedovf	gcc	-O0	なし	L	コンパイル	—	段階
signedovf	gcc	-O0	あり	P	—	—	—
signedovf	gcc	-O1	なし	L	コンパイル	—	段階
signedovf	gcc	-O1	あり	P	—	—	—
signedovf	gcc	-O2	なし	L	コンパイル	—	段階
signedovf	gcc	-O2	あり	P	—	—	—
signedovf	gcc	-O3	なし	L	コンパイル	—	段階
signedovf	gcc	-O3	あり	P	—	—	—

付録 C　拡張図表

本文の記述を補う拡張図表を示す. 表 C.1 は性質カタログ 24 プロパティにおける実装状況の内訳（7 章（3）. implemented 7 のほか partial 1・candidate 2・unimplemented 14），表 C.2 は三法の実現機構の対比（5.2 節），図 C.1 は証拠束の生成と独立検証の分離の構造（5.4 節），図 C.2 は第一喪失点の分布と性質別観測消失率（6.2 節），図 C.3 はオラクルの対照実験の全構成（6.4 節），表 C.3 は非侵襲性・決定性・独立検証の全数サマリ（6.6 節・6.7 節），表 C.4 は独立検証器の検査項目の一覧（5.4 節・6.7 節），表 C.5 は先行研究との対比（3 章），表 C.6 は較正（破損させた入力の検出）の全数（6.7 節），表 C.7 は二分探索の遷移点と観測アンカー（6.3 節）である.

表 C.1　性質カタログ（24 プロパティ）の実装状況

Table C.1　Implementation status of the property catalogue (24 properties).

性質クラス	意味	implemented
must-survive	ソースに書かれた効果が成果物まで残ること	3
must-not-appear	現れてはならない要素が現れないこと	1
must-originate-from	定義が承認された供給元から来ていること	0
must-be-configured	評価済みの構成でビルドされたこと	3
must-remain-unobservable	秘密が成果物から読み取れないこと	0
計	—	7

表 C.2　三法の実現機構の対比

Table C.2　The three attribution methods compared.

方法	用いる機構	性格
1 パス直後の中間表現の観測	-print-after-all	全パスのダンプを取り，対象と対照を同じ規約で数える
2 パス予算の二分	-opt-bisect-limit=N	予算 n で失われ n−1 で残るなら n 番目の（スキップ可能）パスを第一消失として帰属．1 と内部表現を共有しない
3 パスプラグイン	out-of-tree プラグイン（純観測）	コンパイラを改変せず，パスの前後で状態を記録する

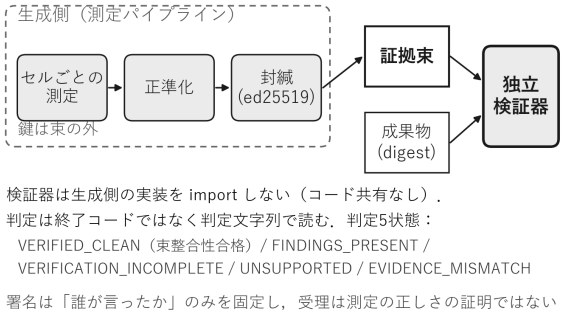


図 C.1　証拠束の生成と独立検証の分離

Figure C.1　Separation of evidence generation and independent verification.

© 2026 Information Processing Society of Japan

性質	保存	前処理	IR Pass	バックエンド	コンパイル過程	観測消失率
authz.assert	8	8	0	0	0	50%
configguard.ifdef	8	8	0	0	0	50%
erasure.wipe	7	0	2	1	6	56.3%
nullcheck.guard	10	0	3	0	3	37.5%
signedovf.guard	9	0	3	0	4	43.8%
計（80セル）	**42**	**16**	**8**	**1**	**13**	—

AST 層・成果物層を第一喪失点とするセルは 0（観測されず）．

観測消失率＝消失セル数／評価セル数（本評価行列内の記述統計．6.2節）

図 C.2　第一喪失点の分布と性質別観測消失率（80 セル）

Figure C.2　Distribution of first-loss stages and per-property observed loss rates.

		clang-18				gcc-13			
		-O0	-O1	-O2	-O3	-O0	-O1	-O2	-O3
名前検索	対象	1	0	0	0	1	0	0	0
名前検索	対照（削除不可）	1	0	0	0	1	0	0	0
ゼロ化命令	対象	1	0	0	0	1	0	0	0
ゼロ化命令	対照（削除不可）	1	2	2	2	1	2	2	2

枠＝盲：名前検索は -O1 以降，削除不可能な対照まで 0（8構成中6）

＝「消えた対象」と「残っている対照」を区別できない

ゼロ化命令：対照は常に非0（盲 0 構成）＝オラクルとして成立する

帰属も動く：名前検索は GlobalOptPass（予算 59）を，呼出し地点

オラクルは DSEPass（予算 50）を名指しする（6.4節）

図 C.3　オラクルの対照実験の全構成（erasure ファミリ 8 構成・6.4 節）

Figure C.3　Oracle control experiment across all eight configurations.

表 C.3　非侵襲性・決定性・独立検証の全数サマリ

Table C.3　Summary of non-invasiveness, determinism and independent verification.

項目	結果	備考
非侵襲性：目的ファイル一致	80/80	観測器の有無で比較（6.6 節）
非侵襲性：実行ファイル一致	80/80	同上
観測器の発火記録	80/80	「不発火での一致」の排除
観測オフ側ビルドに観測器出力がないこと	80/80	不在の確認
決定性：再ビルドのダイジェスト一致	80/80	セル単位
決定性：二度ビルドのプローブ	2/2	
封緘した証拠束	40	ed25519・鍵は束の外
独立検証 VERIFIED_CLEAN（束整合性合格）	40/40	束あたり 15 検査（表 C.4）
較正失敗（破損束の受理）	0	
終了コードと判定文字列の食い違い	0	
署名が覆わない結果ファイル	17	6.7 節・7 章

表 C.4　独立検証器の検査項目

Table C.4　Checks performed by the independent verifier.

ID	検査内容
S0	検証に用いる公開鍵が使用可能であること（ラン全体の前提検査）
S1	束が自己完結していること（symlink・hardlink・fifo・外部への経路がない）
S2	秘密鍵の素材が束の中にないこと
S3	manifest が外部から与えた鍵で検証できる分離署名を持つこと
S4	ディスク上のファイル群が署名済み manifest の列挙とサイズ・sha256 で一致すること
S5	署名済み manifest と evidence の記録が「この束が何か」で一致すること
S6	署名済み run manifest がこの束をこのランに位置づけること
C0	evidence.json が読め，構文解析できること
C0a	schemaVersion が検証器の実装する版であること
C1	artifact.sha256 がディスク上のバイト列と一致すること
C2	evidenceDigest がそれ自身の記録のダイジェストであること
C3	観測点の判定列が単調で，段階帰属が規約表に従うこと
C4	カバレッジが宣言どおりに数えられていること
C5	主張された確信水準が一致の証拠に支えられていること
C6	Pass を主張する性質が Pass を特定し，特定した Pass が記録された集合の中にあること
F1	field notes と未解決項目の参照先が実在すること

S0 はラン全体の前提検査であり，束あたりに実行される検査は S1〜F1 の 15 である（6.7 節）．

表 C.5　先行研究との対比（3 章）

Table C.5　Comparison with prior work.

観点	Yang ら[1]	STACK[12]	Geimer ら[13]	本稿
対象性質	秘密消去	UB 依存の検査	定数時間	宣言語彙（5 性質実測）
ツールチェーン	改造 Clang 3.9	無改造（ソース解析）	無改造 GCC/LLVM	無改造 clang-18・gcc-13
観測の場所	コンパイラ内部（改造 Pass）	ソース（静的）	バイナリ（漏えい検出）	ビルド工程の 6 観測点
消失 Pass の帰属	○（改造で直接）	—	○（フラグ切り分け）	三法一致（8 セル）
前処理段階の帰属	—	—	—	○（16 セル）
証拠束・独立検証	—	—	—	○
実プロジェクト評価	○（11 OSS）	○（Linux 等）	○（BearSSL・MbedTLS）	—（合成 fixture のみ）

D’Silva ら[11]は問題の定式化を与える位置づけであり，システム比較には含めない．○/—は各研究の主対象の対比であり優劣ではない（3 章）．

表 C.5 に含めない近接研究として，性質を witness・注釈として運ぶ系譜[14][15]，コンパイラ拡張で消去等を守る系譜[16]，実コンパイラ由来バグと Pass 起源の実証研究[17][18]がある．いずれも拡張・改造を前提とするか工程帰属と証拠化を扱わず，無改造・帰属・証拠化・独立検証を一つの運用に閉じる本稿の位置は変わらない．

© 2026 Information Processing Society of Japan

表 C.6　較正：破損させた入力が検出されること（6.7 節）

Table C.6　Calibration: corrupted inputs are detected.

較正の対象と破損方法	規模	結果
検証器較正：破損させた束を受理しないこと	40 束	受理誤り 0（全束 PASS）
判読ゲート decoy：対照を対象の複製に置換	1 種	検出（CONTROL_IS_CLONE 他）
判読ゲート decoy：判読の引用 42 行を削除	1 種	検出（QUOTE_NOT_FOUND 他）

表 C.7　二分探索の遷移点と観測アンカー（erasure・clang-18・-O2）

Table C.7　Bisection transition point and observation anchors.

項目	値	備考
総 Pass 数（-S -emit-llvm）	94	遷移予算 50
総 Pass 数（-S）	138	遷移予算 50
予算 49 のビルド	wipe 残存	遷移点の直前
予算 50 のビルド	wipe 消失	50 番目＝DSEPass
遷移の単調性	8/8 セル	表 5・6.3 節

付録 D　fixture の最小コード例

測定に用いた fixture の代表として，erasure（memset による秘密消去）の測定対象ソースの抜粋をリスト D.1 に示す．対象（subject）・オラクル対照（削除不可能）・生存対照（推奨される修正形）の 3 関数が同一翻訳単位に同居し，manifest で宣言される（6.1 節）．秘密の生成（get_secret）と消費（consume）は別翻訳単位の不透明な関数であり，測定対象そのものが退化して消えることを防ぐ．掲載例は教科書的な既知パターンのみからなり，実在システムの秘密も攻撃経路も含まない(研究倫理(3))．コメントは和訳し，宣言部は省略した．対応する生成アセンブリの before/after 抜粋をリスト D.2 に示す．

リスト D.1　erasure fixture の測定対象ソース（抜粋）

Listing D.1　Subject source of the erasure fixture (excerpt).

```
/* 対象（subject）：wipe はバッファの最後の使用であり，
   デッドストアとして削除され得る */
void handle_request(void) {
    unsigned char secret[32];
    get_secret(secret, sizeof secret);
    consume(secret, sizeof secret);
    memset(secret, 0, sizeof secret);
}
/* オラクル対照：wipe の後に読み取りがあるため，どの
   水準でも削除できない．-O1 以降はインライン化された
   ゼロ書き込みになる（規約 1 の根拠） */
void wipe_kept(void) {
    unsigned char secret[32];
    get_secret(secret, sizeof secret);
    memset(secret, 0, sizeof secret);
    consume(secret, sizeof secret);
}
/* 生存対照（推奨される修正形・VG-MEM-006）：最後の
   使用でも削除が許されない消去 */
void handle_request_bzero(void) {
    unsigned char secret[32];
    get_secret(secret, sizeof secret);
    consume(secret, sizeof secret);
    explicit_bzero(secret, sizeof secret);
}
```

リスト D.2　erasure fixture の生成アセンブリ抜粋（clang-18・x86-64）

Listing D.2　Generated assembly of the erasure fixture (excerpt).

```
# -O0：wipe は call memset として存在（cfi 等の指示行は省略）
handle_request:
    leaq  -32(%rbp), %rdi
    callq get_secret@PLT
    leaq  -32(%rbp), %rdi
    callq consume@PLT
    leaq  -32(%rbp), %rdi
    xorl  %esi, %esi
    movl  $32, %edx
    callq memset@PLT
    retq
# -O2：subject では wipe が消失（DSEPass への帰属．表 5）
handle_request:
    movq  %rsp, %rbx
    callq get_secret@PLT
    callq consume@PLT
    retq            # memset は現れない
# -O2：削除不可の対照はインラインゼロ化として残存．シンボル
# memset は出現しない＝名前検索が判定不能となる（規約 1・表 6）
wipe_kept:
    callq get_secret@PLT
    xorps %xmm0, %xmm0
    movaps %xmm0, 16(%rsp)
    movaps %xmm0, (%rsp)
    callq consume@PLT
    retq
```

付録 E　追加実験（帰属妥当性と証拠被覆）

本付録は，外部レビューの指摘を受けて数値凍結後に追加した 4 つの検査である．いずれも凍結済み測定ツリーと同一のツールチェーン上で実施し，出力は測定結果ディレクトリの外に保存した．本付録の数値は本文の凍結マクロ体系の外にある．

E.1　チャネル単独の帰属．三法の各チャネル（ダンプ走査・二分・プラグイン）が単独で特定する Pass を，凍結済みの trace 記録からチャネルごとに別々に再計算した．いずれのチャネルも，Pass を特定できたのは三法一致で帰属が確定した 8 セル（6.3 節）と同一の集合であり，特定した Pass も全セルで一致した（単独一致 8/8/8・チャネル間の食い違い 0 件・二法どまり 0）．本測定の範囲では，三法一致の要求によって帰属から漏れたセルはなく，三法の増分価値は観測された救済ではなく，単独チャネルの故障を検出するための冗長性である．

E.2　帰属 Pass の抑止による因果介入．公式の -print-pipeline-passes で得たパイプライン指定から帰属 Pass を除き，同一ツールチェーンの opt・llc に与えて再コンパイルし，防御の生存を中間表現と最終アセンブリの両方で測った（帰属確定 8 セル・表 E.1．パイプライン指定の再構成に基づく介入であり，コンパイラ内部の起動と完全同一ではない）．nullcheck の 3 セルは EarlyCSEPass の除去だけで null 検査が最終アセンブリまで復活し，帰属 Pass の因果性が確認された．erasure の 2 セルは DSEPass の除去で memset が中間表現に復活するが，バックエンドが再び除去し，アセンブリは介入前とバイト同一に留まった．signedovf の 3 セルは EarlyCSEPass を除去しても中間表

© 2026 Information Processing Society of Japan

現の段階で既に消失していた．すなわち，第一喪失点の帰属 Pass は「除去すれば防御が戻る唯一の原因」を意味しない（付録 B の定義の実験的裏付け）．

表 E.1　帰属 Pass の抑止介入の結果

Table E.1　Outcomes of suppressing the attributed pass.

性質（セル数）	除去した Pass	中間表現	最終アセンブリ
nullcheck（3）	EarlyCSEPass	復活	復活（検査が再出現）
erasure（2）	DSEPass	復活	再消失（バイト同一）
signedovf（3）	EarlyCSEPass	復活せず	復活せず

E.3　対象束縛オラクルの試作. 7 章(16)の欠陥に対し，中間表現テキストの def-use を辿って対象バッファへの到達バイトを数える試作オラクルを実装し，4 種の最小ミュータントで検査した（表 E.2）．無関係バッファのみをゼロ化する decoy は素朴な計数では検出できないが，束縛オラクルは BOUND_ABSENT と判定する．ただし，本試作は対象バッファ名の指定を前提とし，指定が無ければ最大の alloca を対象に選ぶため decoy を誤判定する．一般のコードへの適用には対象選択の実装が先行する．

表 E.2　対象束縛オラクル試作の判定

Table E.2　Verdicts of the prototype target-bound oracle.

ミュータント	束縛判定（到達/対象バイト）	素朴な計数
完全ワイプ	BOUND_PRESENT（32/32）	1
無関係バッファのみゼロ化	BOUND_ABSENT（0/32）	1
半分ワイプ	BOUND_PARTIAL（16/32）	1
ループでゼロ化	UNSUPPORTED_CONSTRUCT	2

E.4　束外結果ファイルへの署名サイドカー. 検証時点で束の署名が覆っていない結果ファイル（6.7 節）に対し，束の外の全結果ファイル 27 件（6.7 節の 17 件に，検証より後に書かれた記録を加えたもの）の sha256 を列挙した manifest を作り，束の封緘と同一の ed25519 鍵で署名した．manifest と署名は結果ディレクトリの外に置き，封緘プロトコルは変更しない．検証では署名が有効で，27 件全てのハッシュが一致した（不一致 0）．署名時点より後に書かれるファイルを覆わない限界は残る．なお，本サイドカーの初版は境界判定の誤りで 1 件（束の索引ファイル）を被覆から漏らしており，本節の数値は修正後の再署名の値である．

付録 F　実装の全体像（VibeGuard Compiler）

本稿の測定は 5 章の実現機構（公式フラグ・純観測プラグイン・スクリプト群）で成立する．その背後で著者が実装してきたコンパイラ検査基盤 VibeGuard Compiler の実装済みの範囲を，実績として表 F.1 に示す（2026-08-18 時点．本付録の数値も本文の凍結マクロ体系の外にある）．

表 F.1　VibeGuard Compiler の実装コンポーネント

Table F.1　Implemented components of VibeGuard Compiler.

構成要素	役割
ドライバ（driver）	ポリシー検証・ツールチェーンのバイナリ digest ピン・性質保存フォールバック（消失時は承認済み低最適化で再コンパイル）
AST ゲート（clang-plugin）	字句知見を AST で確認・棄却・精緻化して要求に変換．プラグイン有無で目的コードの digest 一致を要求
IR チェックポイント（llvm-pass）	最適化前後の状態判定．構造再解析と metadata 注釈の二重表現を並置し食い違いを可視化
Pass 観測器・導入検出（pass-instrumentation）	（Pass，IR 単位）の除去履歴の全記録．逆方向の導入物検出と対象名解決の記録
ELF 検証器（elf-verifier）	成果物内要素の由来分類（未解決を説明済みに丸めない）．構造フィールド直読
リンク記録（link-wrapper）	リンク 1 回の記録と方針照合．呼出し側供給のマップを拒否（証拠の自己供給防止）
証拠・独立検証（evidence）	正準化・封緘と，正準化器を共有しない再導出による検証（5.4 節の系）
署名・由来（provenance）	ed25519 分離署名・ビルド由来記述・再現ビルド検査
構成包絡（envelope）	測定済み構成での性質不成立割合の算出．包絡キーの名目性への防御
汎用指紋（fingerprint）	性質非依存の IR 正規化．クロスレベル安定性は否定的結果として記録
評価ハーネス（eval）	アブレーション・較正・比較・メタモルフィック・陰性対照・第二ベンダの 6 レーン
静的検査ツール（製品）	v0.3.6・85 検出ルールを出荷済み．うち 2 ルールは本稿の閉ループ由来（6.8 節）

これらは出荷物と測定系を分離する境界規則の下で管理され，本稿の非侵襲性の主張（5.1 節）に影響しない．評価ハーネス各レーンの測定値は本稿の凍結数値体系の外にあるため，本稿では数値として報告しない．

© 2026 Information Processing Society of Japan